ASCENT INTELLIGENT
TECHNOLOGIES

Ascent Intelligent Technologies Private Limited
Gachibowli, Hyderabad – 500032

TECHNICAL & OPERATIONS

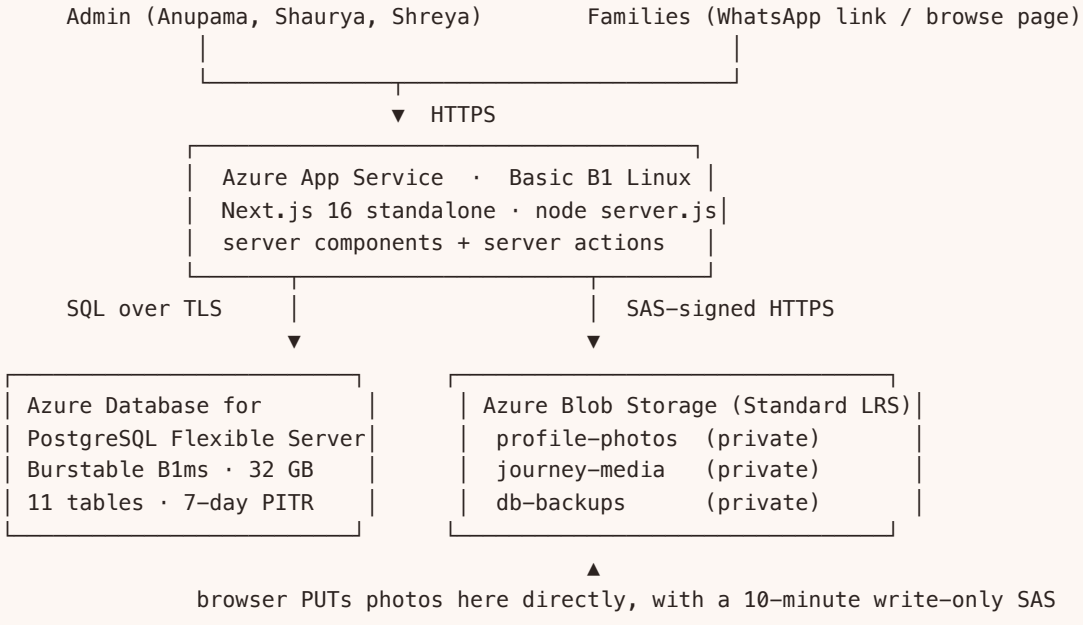
PLATFORM **AnuRupa Matrimony**
VERSION **1.0**
DATE **13 October 2026**

How the platform is built, how it is configured, how a change reaches production, how to put the previous version back, and how the data is protected and recovered. Written for whoever runs it next — a developer who has never seen the code should be able to operate it from this document and the repository alone.

> **Where things are.** Production: `https://anurupa-8c72e8.azurewebsites.net`. Source: GitHub, `Ascent-Intelligent-Techonologies/match_making_app`, branch `main`. Azure: subscription *Azure subscription 1*, resource group `anurupa-rg`, region Central India. Everything operational lives under `azure/` in the repository; `azure/README.md` is the hands-on runbook and this document is its context.

1 Architecture

A single Next.js application, server-rendered, with one managed PostgreSQL database and one storage account. There is no separate API tier, no queue, no cache server and no container orchestration: at a few thousand profiles and a consultant team of three, each of those would be cost and surface area with nothing to pay for it.



COMPONENT	WHAT IT IS, AND WHY
Application	Next.js 16 on Node 22, TypeScript, Tailwind. Server components render every page; server actions handle every write, with Zod validation on the server, so no business rule depends on the browser. Deployed as the *standalone* bundle (≈ 17 MB zipped) and started with `node server.js`.
Hosting	App Service plan **Basic B1** (1 core, 1.75 GB), Linux. Always On, HTTPS only, TLS 1.2 floor, FTPS off, container logging on. The smallest tier that keeps a process warm; the next step up is one command (§4.4).
Database	PostgreSQL 17, **Burstable B1ms**, 32 GB storage, 7-day automated backups. The free-tier SKU for twelve months on a new account. All access is through the application's own connection pool over enforced TLS; the browser never talks to the database.
Data access	One module, `src/lib/db/`: a `pg` pool with type parsers (`client.ts`) and a table-agnostic CRUD layer (`crud.ts`) that every data module in `src/lib/data/` is written against. Values are always bound parameters; identifiers are pattern-checked; `updateMany` and `deleteMany` refuse to run without a filter. Three hand-written statements remain, each commented as to why.
Storage	Standard LRS, hot tier. Three private containers. Nothing is reachable without a SAS token the server mints per render, valid one hour, computed locally (an HMAC — no round trip). Deleted blobs are kept 14 days.
Photos	Each photo is three blobs: the original (kept, never shown), a 480 px WebP *thumb* for grids and cover tiles, and a 1200 px WebP *medium* for the carousel, slideshow and share page. Both copies are made once, at upload, by sharp; pages serve them directly. The server never resizes on request.
Uploads	Straight from the browser to Blob Storage. The server issues a write-only SAS for a path it chose; the browser PUTs; the server then reads the blob back inside the datacenter, confirms it is an image, writes the two copies and records the row. A failure at any step deletes what was written. The App Service never buffers an upload.
Identity & sessions	One administrator password (bcrypt hash in configuration); a signed, HTTP-only session cookie, 12 hours. Families are identified by name and phone, held in a signed cookie, matched to the client book by normalised phone number.
Deployment	GitHub Actions on every push to `main`, signing in to Azure with OpenID Connect through a user-assigned managed identity that holds Contributor on the web app alone. No credential is stored anywhere.
Development backup	Vercel, branch `vercel-dev` — the last Supabase-based commit. Not production; `vercel.json` on `main` stops Vercel building it.

1.1 Security model

- **Network.** The database firewall admits Azure-internal sources (the `0.0.0.0` sentinel) and the operator's IP; the public internet is refused before any password is checked. Public access can be replaced with a private endpoint (≈ ₹700/month) if a contract ever demands no public endpoint.
- **Data.** Row-level security is enabled on every table with no policies; the application connects as the table owner, which bypasses it; any other role is denied everything.

- **Media.** Private containers; one-hour SAS for reads; ten-minute create/write-only SAS scoped to a single server-chosen path for uploads; CORS limited to the site's origin.
- **Disclosure.** Share links carry one of three access levels enforced field by field on the server. Finances, internal tags, Aura and Urgent markers, potential-client markers, the middleman's details and private notes are never serialised to a client page at any level.
- **Secrets.** Live only in App Service configuration (encrypted at rest, RBAC-protected) and in the operator's git-ignored `azure/.env.azure`. Not in the repository, not in GitHub, not in CI logs.

1.2 Repository map

PATH	CONTENTS
`src/app/`	Routes. `admin/(protected)/…` is the consultants' area; `share/[token]` and `browse` are what families see.
`src/lib/data/`	One module per concern (profiles, photos, clients, share-links, shortlists, searches, followups, team-notes, journey, settings). All SQL goes through `src/lib/db/`.
`src/lib/actions/`	Server actions — the only way the browser changes anything.
`src/lib/storage/blob.ts` · `src/lib/images.ts`	Blob Storage and the derivative pipeline.
`supabase/setup.sql` · `supabase/migrations/`	The whole schema in one re-runnable file, and the versioned history that produced it. Plain PostgreSQL; the directory name is where the project started, not where it runs.
`azure/`	`infra.sh` (provision), `deploy-local.sh`, `backup-db.sh`, `restore-db.sh`, `migrate-from-supabase.mjs`, `backfill-thumbnails.mjs`, and `README.md`, the runbook.
`.github/workflows/deploy.yml`	Build on pull requests; build, deploy and smoke-test on pushes to `main`.
`docs/`	This document, the user guide, quotation, invoice, cost comparison and competitive analysis, each as HTML and PDF.

2 Configuration

The application reads everything from environment variables, so nothing about where it runs is in the code. In production those are App Service *application settings*; locally they are `.env.local`. `azure/infra.sh` writes the infrastructure-derived ones, and `azure/deploy-local.sh` pushes the application secrets from the operator's `.env.local`.

SETTING	PURPOSE · SET BY
DATABASE_URL	`postgresql://…?sslmode=require`. infra.sh (it generates the password; the only copy is `azure/.env.azure`).
AZURE_STORAGE_ACCOUNT · AZURE_STORAGE_KEY	Blob Storage; the key signs every SAS. infra.sh.
ADMIN_PASSWORD_HASH	bcrypt hash of the administrator password. deploy-local.sh, from `.env.local`. Generate with `node -e "console.log(require('bcryptjs').hashSync('…', 12))"` and escape every `$` as `\$` in `.env.local`.
SESSION_SECRET	Signs admin and family session cookies. Rotating it signs everyone out. deploy-local.sh.
NEXT_PUBLIC_SITE_URL	Base of every share link. **Read at build time**, not run time — change it in the GitHub variable `SITE_URL` (and `.env.local`) and redeploy; changing only the App Service setting does nothing.
HOSTNAME=0.0.0.0	The standalone server binds to `process.env.HOSTNAME`, and App Service sets that to a machine name it cannot bind to. Without this the app starts and refuses every request. infra.sh.
WEBSITES_CONTAINER_START_TIME_LIMIT=1800	Extracting the bundle and cold-starting Node on a B1 can exceed the 230-second default. infra.sh.
SCM_DO_BUILD_DURING_DEPLOYMENT=false	The bundle arrives built; App Service must not try to build it again. infra.sh.
PGSSL_NO_VERIFY=1 (optional)	Disables certificate verification for a self-hosted database. Never set on Azure.
PGPOOL_MAX (optional)	Connection pool size, default 10. B1ms allows a few dozen in total.

GitHub repository *variables* (not secrets — none is confidential): `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`, `AZURE_WEBAPP_NAME`, `SITE_URL`.

```
az webapp config appsettings list -g anurupa-rg -n anurupa-8c72e8 -o table      # see
everything
az webapp config appsettings set  -g anurupa-rg -n anurupa-8c72e8 --settings KEY=value
./azure/deploy-local.sh --settings-only                                          # push
.env.local's secrets
```

Any change to application settings restarts the app; expect thirty seconds of unavailability.

3 Deployment

3.1 From a push to main (normal)

1. Merge or push to `main`. Pull requests only build and type-check; they never deploy.
2. GitHub Actions, on a Linux runner: `npm ci`, generate route types, type-check, lint, `next build` as a standalone bundle with `NEXT_PUBLIC_SITE_URL` baked in, zip (under UTC), publish the artifact.

3. The deploy job signs in with OIDC, zip-deploys to the web app, then polls `/browse` for up to ten minutes and **fails the run if the site does not answer** — App Service itself reports success as soon as the zip lands, before the container has restarted.
4. Watch it: `gh run watch`, or *Actions* on GitHub. A deploy takes four to eight minutes end to end.

3.2 From a laptop (when GitHub is unavailable, or for a quick fix)

```
./azure/deploy-local.sh               # build, push .env.local secrets, deploy, poll
./azure/deploy-local.sh --no-build    # redeploy the last build
./azure/deploy-local.sh --skip-settings
```

Identical result, except that a Mac-built bundle runs sharp on its WebAssembly fallback (slower resizing on upload; the CI build uses the native Linux binary). Prefer the pipeline.

3.3 Database changes

Schema changes are SQL files in `supabase/migrations/`, numbered and re-runnable, and are applied by hand with `psql "$DATABASE_URL" -f supabase/migrations/NNN_….sql` **before** the code that needs them is deployed. `supabase/setup.sql` is the whole schema in one file for a new environment, and must be kept in step. Take a backup first (§5.2).

3.4 Scaling

```
az appservice plan update -g anurupa-rg -n anurupa-plan --sku B2            # 2 cores, 3.5
GB
az postgres flexible-server update -g anurupa-rg -n anurupa-pg-8c72e8 \
    --sku-name Standard_B2s --tier Burstable                                 # in place; a
restart, no migration
az postgres flexible-server update -g anurupa-rg -n anurupa-pg-8c72e8 --storage-size 64
# grows; never shrinks
```

At the volumes in the cost sheet none of this is needed; the figures hold to roughly 25,000 profiles. Storage of photographs is not a scaling concern at all — ten thousand HD photos is about 50 GB and about ₹100 a month.

4 Rollback

Rollback means putting the previous working version back after a deploy turns out to be wrong. On this platform the application and the data roll back separately, and the application side is quick because every deploy is a complete, self-contained bundle.

4.1 Application — three ways, fastest first

SITUATION	DO THIS
Last deploy is bad; the one before was fine, within 7 days	Re-run the previous green run's deploy job. Its artifact is still there, so nothing is rebuilt: `gh run list --branch main --limit 5` → note the id of the last good run → `gh run rerun <id>` (or in GitHub: the run → *Re-run all jobs*). Five minutes.
The bad change should be undone in history too	`git revert <bad-commit>` and push. The pipeline deploys the reverted code. This is the one to prefer when others will keep working on `main`.
GitHub is unavailable	`git checkout <last-good-commit>` then `./azure/deploy-local.sh`. Afterwards return to `main`.

In every case, confirm with the smoke test the pipeline does: `curl -sI https://anurupa-8c72e8.azurewebsites.net/browse` should return `200`, and the admin should be able to sign in.

4.2 Configuration

A bad application setting is corrected with `az webapp config appsettings set` (§2); the app restarts on its own. There is no history of settings in Azure, so the values to restore are in `azure/.env.azure` and `.env.local` — keep those files.

4.3 Database

Migrations are **forward-only**: there are no down-scripts, because a column that has held data cannot be un-added without losing it. If a migration must be undone, write a new migration that reverses it. For the *data* — a bad bulk import, a cleanup that went too far — use point-in-time restore (§5.3), which returns the database to any minute of the last seven days.

> **Order matters.** If a release included a migration, roll the application back *first*; the old code tolerates extra columns, but new code cannot run against the old schema.

5 Backup & restore

5.1 What is protecting the data, today

DATA	PROTECTION	RECOVERS FROM
Database	**Point-in-time restore**, automatic, every minute of the last **7 days**, included with the server.	Bad import, accidental delete, bad migration — "take me back to 10:40 this morning".
Database	**Portable dump** (`azure/backup-db.sh`) to the private `db-backups` container. On demand; take one before any migration, import or cleanup.	Server deleted; moving to another server or cloud; keeping a copy older than 7 days.
Photos & media	**Soft delete**, 14 days, for blobs and containers. Storage is locally redundant (three copies in the datacenter).	A photo or a whole container deleted by mistake, within 14 days.
Code & configuration	Git (GitHub); `azure/.env.azure` and `.env.local` on the operator's machine.	Anything; a whole environment can be rebuilt (§5.5).

> Nothing here is off-site in the geographic sense: all of it is in Azure Central India. For a matchmaking consultancy that is the right trade; if a copy outside Azure is ever wanted, a weekly `./azure/backup-db.sh --keep-local` and a copy of `azure/backups/` to any drive is it.

5.2 Take a backup

```
./azure/backup-db.sh                # pg_dump → db-backups/anurupa-<UTC time>.dump, lists
what is stored
./azure/backup-db.sh --keep-local   # and keep the file in azure/backups/
```

Custom format, compressed, about 330 KB at today's size. Recommended cadence: before every migration or import, and weekly otherwise. There is no scheduler for it yet; a calendar reminder is honest about that.

5.3 Restore the database to a point in time (preferred for "undo")

```
# 1. a NEW server, restored to the moment you choose (UTC); 10–20 minutes
az postgres flexible-server restore -g anurupa-rg --name anurupa-pg-restored \
    --source-server anurupa-pg-8c72e8 --restore-time "2026-10-13T05:50:00Z"
# 2. same firewall rules as the original (infra.sh's "Database firewall" step, or the
portal)
# 3. point the app at it, then restart
az webapp config appsettings set -g anurupa-rg -n anurupa-8c72e8 \
    --settings DATABASE_URL="postgresql://anurupa_admin:<password>@anurupa-pg-
restored.postgres.database.azure.com:5432/anurupa?sslmode=require"
# 4. update azure/.env.azure to match; delete the old server once satisfied
```

The restored server keeps the original's admin password. Nothing on the original is changed by this, so it can be compared first.

5.4 Restore the database from a dump

```
./azure/restore-db.sh                                  # list backups
./azure/restore-db.sh anurupa-2026-10-13T06-38-41Z.dump  # into production — asks you to
type the host name
./azure/restore-db.sh <name> --into "postgresql://…"    # into any other database, e.g. a
rehearsal
```

```
az webapp restart -g anurupa-rg -n anurupa-8c72e8        # afterwards, so the pool
reconnects
```

The restore is one transaction: either the whole backup lands or nothing changes. **This procedure was rehearsed before being documented**: the first backup was restored into a scratch database on the same server and matched production row for row (2,783 profiles, 11 clients, 23 photos, 9 links).

5.5 Recover a deleted photo, or rebuild everything

```
# a photo deleted in the last 14 days (all three of its blobs)
az storage blob undelete --account-name anurupast8c72e8 -c profile-photos -n "<profile-
id>/<uuid>.jpg"
az storage blob undelete --account-name anurupast8c72e8 -c profile-photos -n "<profile-
id>/<uuid>.thumb.webp"
az storage blob undelete --account-name anurupast8c72e8 -c profile-photos -n "<profile-
id>/<uuid>.medium.webp"
# the row in profile_photos is restored with the database (5.3 or 5.4)

# a whole new environment, from nothing
./azure/infra.sh                        # resources + schema
./azure/restore-db.sh <dump>            # data
azcopy copy "https://<old>.blob.core.windows.net/profile-photos?<sas>" \
            "https://<new>.blob.core.windows.net/profile-photos?<sas>" --recursive   #
media
./azure/deploy-local.sh                 # application
```

6 Monitoring & day to day

```
az webapp log tail -g anurupa-rg -n anurupa-8c72e8                # live application log
az webapp restart  -g anurupa-rg -n anurupa-8c72e8
az postgres flexible-server show -g anurupa-rg -n anurupa-pg-8c72e8 --query state
az monitor metrics list --resource <web app id> --metric CpuTime,MemoryWorkingSet,Http5xx
```

- The pipeline's smoke test is the first line of monitoring: a deploy that leaves the site unreachable is a red run, not a quiet outage.
- App Service *Diagnose and solve problems* in the portal explains most restarts without reading logs.
- Costs: about ₹1,350/month for the first year (database free tier), about ₹3,470 after. Check *Cost Management* in the first week to confirm the free-tier credit is being applied.
- The free database tier ends twelve months after the account was created; nothing changes technically, the bill does.

7 Quick reference

I WANT TO…	COMMAND
Deploy	push to `main` · or `./azure/deploy-local.sh`
Roll back the app	`gh run rerun <last good run id>` · or `git revert` + push
Change a setting	`az webapp config appsettings set … --settings K=V`
Back up the database	`./azure/backup-db.sh`
Undo data changes	point-in-time restore, §5.3
Restore from a dump	`./azure/restore-db.sh <name>`
Recover a deleted photo	`az storage blob undelete …`, §5.5
See logs	`az webapp log tail -g anurupa-rg -n anurupa-8c72e8`
Apply a migration	backup, then `psql "$DATABASE_URL" -f supabase/migrations/NNN.sql`
Rebuild from nothing	`./azure/infra.sh` → `restore-db.sh` → copy blobs → `deploy-local.sh`
Turn everything off	`az group delete --name anurupa-rg --yes` — irreversible